

```
$ node breakfast.js --mode=async
```

JavaScript 非同步

// 用一頓早餐，搞懂 Promise 與 async/await

sync

排隊等待

promise

未來值容器

await

同步的外表

event loop

幕後調度

\$ cat menu.md

今日菜單

01 / 早餐比喻

同步阻塞 vs 非同步開工

02 / Promise

三種狀態、then/catch、逾時保護

03 / async & await

語法糖、串行 vs 並行

04 / Event Loop

VIP 通道與執行順序測驗

目標：看懂任何一段 async 程式碼的「實際執行順序」。

PART 01

早餐比喻

同步：等咖啡煮好才烤吐司

同步：主執行緒被鎖死

```
1  function makeCoffee() {  
2      console.log("開始煮咖啡...");  
3      const start = Date.now();  
4      while (Date.now() - start < 2000) {} // 阻塞！  
5      console.log("咖啡好了！");  
6  }  
7  
8  function prepareBreakfastSync() {  
9      makeCoffee(); // 2s  
10     makeToast(); // 1.5s  
11     fryEgg();     // 1s  
12 }
```

CONSOLE - 4.5s

開始煮咖啡...

咖啡好了！

開始烤吐司...

吐司烤好了！

(這段期間頁面完全卡死)

非同步：全部同時開工

```
1  function makeCoffee() {  
2      return new Promise((resolve) => {  
3          console.log("開始煮咖啡...");  
4          setTimeout(() => {  
5              resolve("咖啡好了！");  
6          }, 2000); // 交給背景計時  
7      });  
8  }  
9  
10 async function prepareBreakfastAsync() {  
11     const coffee = await makeCoffee();  
12     console.log(coffee);  
13     // ...吐司、蛋  
14 }
```

CONSOLE - ~2s

開始煮咖啡...

開始烤吐司...

開始煎蛋...

咖啡好了！吐司烤好了！蛋煎好了！

主執行緒全程沒有被卡住

親眼比一次

```
$ node breakfast.js
```

選一種方式開始做早餐

同步 vs 非同步：早餐賽跑

▶ 同步（一件做完才做下一件）

▶ 非同步（全部同時開工）

☕ 煮咖啡 (2s)

🍞 烤吐司 (1.5s)

🍳 煎蛋 (1s)

同步總時間 = $2 + 1.5 + 1 = 4.5$ 秒； 非同步總時間 = $\max(2, 1.5, 1) = 2$ 秒

PART 02

Promise

一個裝著「未來值」的容器

三種狀態，單向變化

pending

待定：還在煮

fulfilled

已解決：resolve("成功")

rejected

已拒絕：reject("失敗")

```
1 let myPromise = new Promise((resolve, reject) => {  
2   let success = true;  
3   setTimeout(() => {  
4     if (success) resolve("成功!");  
5     else reject("失敗!");  
6   }, 2000);  
7 });  
8  
9 myPromise.then(result => console.log(result))  
10 .catch(error => console.log(error));
```

逾時保護：Promise.race + AbortController

```
1  const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
2
3  const fetchWithTimeout = (url, { timeout = 3000 } = {}) => {
4    const controller = new AbortController();
5    const timer = delay(timeout).then(() => controller.abort());
6
7    const request = fetch(url, { signal: controller.signal });
8
9    return Promise.race([request, timer]).catch((error) => {
10      if (error.name === "AbortError") return Promise.reject("請求逾時");
11      return Promise.reject(error.message || "未知錯誤");
12    });
13  };
```

Promise.race

誰先完成聽誰的 —— 拿來做逾時

AbortController

真的取消請求，不浪費資源

PART 03

async / await

非同步的裡子，同步的外表

then 鏈 → await

```
1  async function loadSpells() {  
2    try {  
3      const res = await fetch("https://hp-api.onrender.com/api/spells");  
4      const data = await res.json();  
5      console.log("spells", data.length);  
6    } catch (err) {  
7      console.error(err.message);  
8    }  
9  }
```

兩者**完全等價**：await 只是把 then 的後續「攤平」成同步的樣子。長流程用 async/await，簡單鏈結用 then。

串行：有相依才排隊

```
1  async function serial() {  
2    const coffee = await makeCoffee();  
3    const toast = await makeToast();  
4    return [coffee, toast];  
5  }
```

TOTAL TIME

≈ 所有任務時間總和 (2 + 1.5 = 3.5s)

適用

先拿 token 再打 API —— 後者依賴前者

並行：沒相依就同開

```
1  async function parallel() {  
2    const [coffee, toast] = await Promise.all([  
3      makeCoffee(),  
4      makeToast(),  
5    ]);  
6    return [coffee, toast];  
7  }
```

TOTAL TIME

≈ 最慢任務 (max(2, 1.5) = 2s)

適用

多支互不相干的列表 API

Promise.all vs allSettled

	Promise.all	Promise.allSettled
一個失敗時	立刻 reject，全軍覆沒	照樣等所有完成
回傳	值的陣列	<code>{status, value/reason}</code> 陣列
適合	缺一不可的資料	「盡量收集」的儀表板

\$ 面試常考：all 是「全有或全無」，allSettled 是「盡力而為」。

PART 04

Event Loop

微任務是 VIP 通道，宏任務排普通隊

幕後調度：兩條佇列



微任務 (VIP)

每回合先清空整條佇列

宏任務 (普通)

微任務全空後，一次取一個

執行順序測驗

```
1 console.log("1");  
2 setTimeout(() => console.log("2"), 0);  
3 Promise.resolve().then(() => console.log("3"));  
4 console.log("4");
```

ANSWER

- 1 → 4 (同步代碼先跑完, Call Stack 清空)
- 3 (微任務 VIP 通道優先)
- 2 (宏任務最後, 即使延遲是 0)

輸出順序 : 1 → 4 → 3 → 2

今日重點

非同步

耗時工作交給背景，主流程不阻塞

Promise

未來值容器：pending → fulfilled / rejected

串行 vs 並行

有相依才 await 排隊，沒相依用 Promise.all

Event Loop

微任務全清空，才輪到宏任務

非同步的總時間 = 最慢的那件事，不是所有事的總和。

Breakfast served.

下一步：fetch 與 AbortController 的實戰練習題

```
$ node breakfast.js --mode=async
```

```
promise → await → race → event loop
```